

Insertion Sort

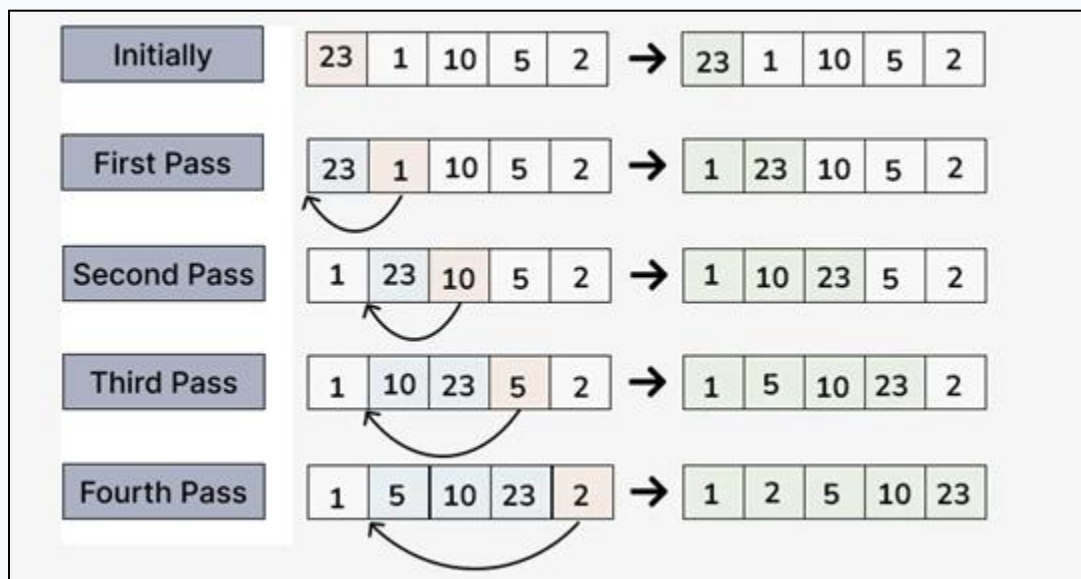
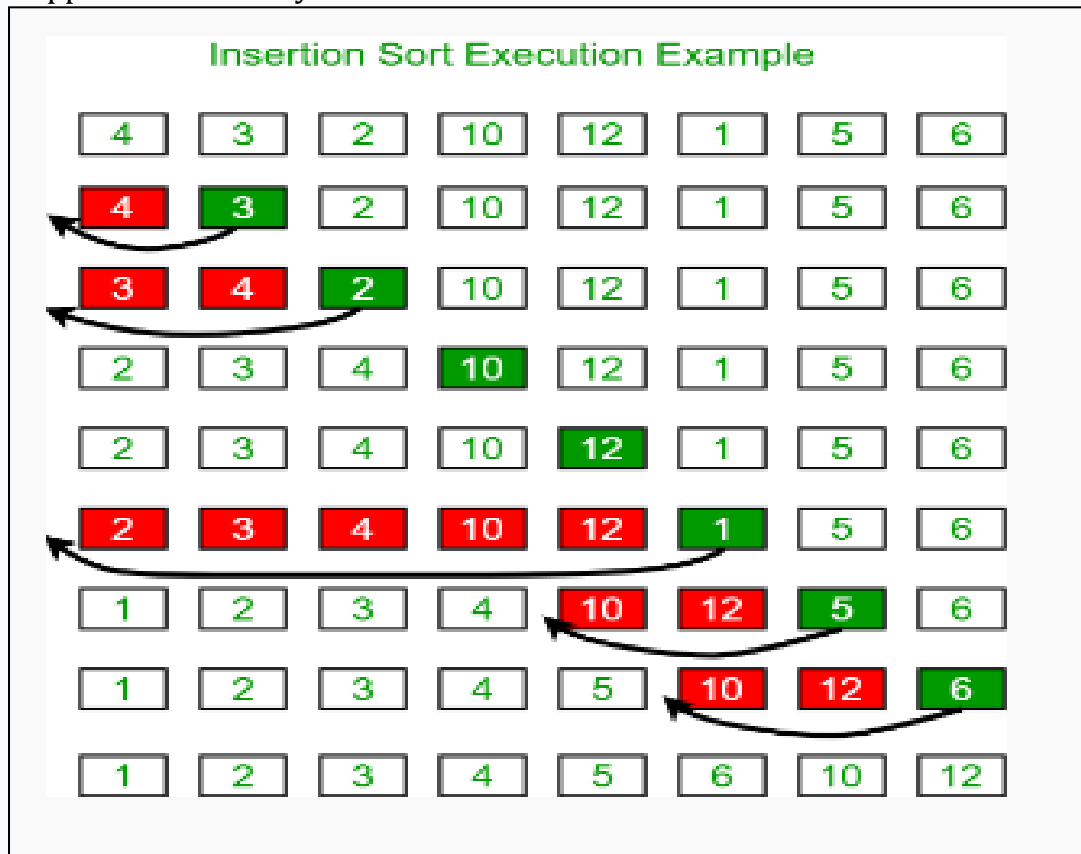
Insertion sort is a [sorting algorithm](#) that places an unsorted element at its suitable place in each iteration.

Insertion sort works similarly as **we sort cards in our hand in a card game**.

We assume that the first card is already sorted then, we select an unsorted card. If the unsorted card is greater than the card in hand, it is placed on the right otherwise, to the left.

In the same way, other unsorted cards are taken and put in their right place.

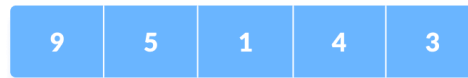
A similar approach is used by insertion sort.



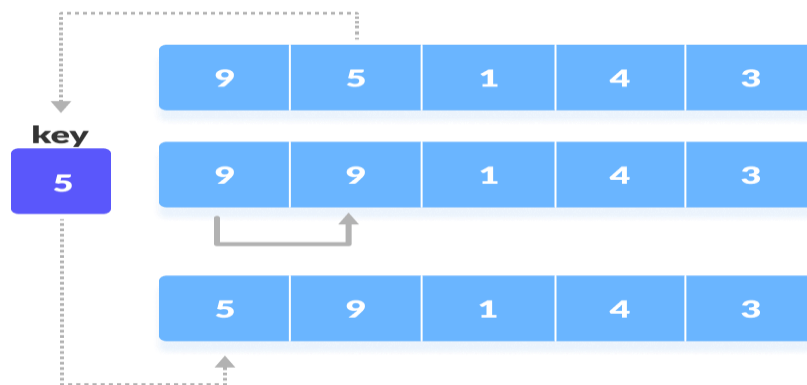
Working of Insertion Sort

The first element in the array is assumed to be sorted. Take the second element and store it separately in key. Compare key with the first element. If the first element is greater than key, then key is placed in front of the first element.

Initial Array/List:



step = 1

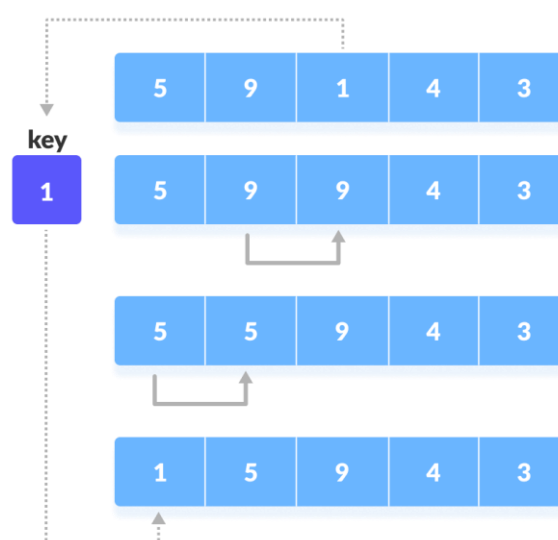


If the first element is greater than key, then key is placed in front of the first element.

Now, the first two elements are sorted.

Take the third element and compare it with the elements on the left of it. Place it just behind the element smaller than it. If there is no element smaller than it, then place it at the beginning of the array.

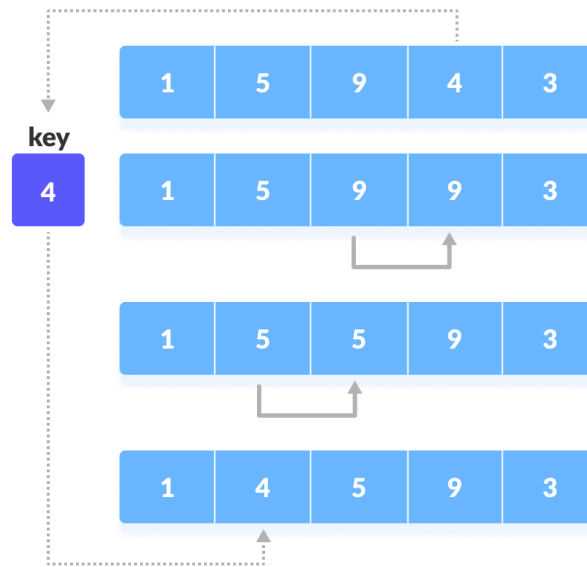
step = 2



Place 1 at the beginning

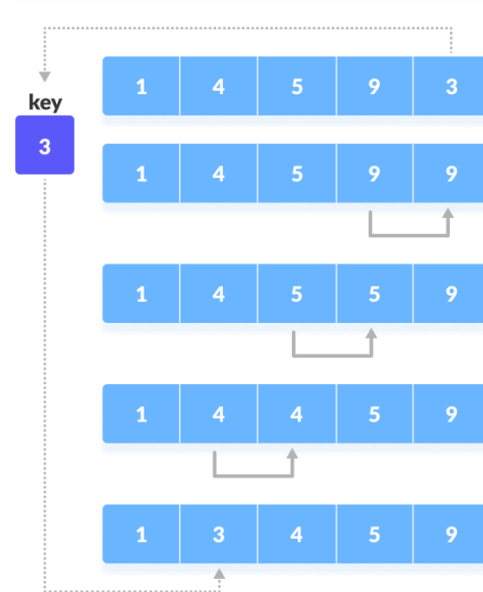
Similarly, place every unsorted element at its correct position.

step = 3



Place 4 behind 1

step = 4



Insertion Sort Complexity

Time Complexity		
Best Case		$O(n)$
Average Case		$O(n^2)$
Worst Case		$O(n^2)$
Space Complexity		$O(1)$

Time Complexities

Worst Case Complexity: $O(n^2)$

- Suppose, an array is in ascending order, and you want to sort it in descending order. In this case, worst case complexity occurs.
- Each element has to be compared with each of the other elements so, for every n th element, $(n-1)$ number of comparisons are made.
- Thus, the total number of comparisons = $n*(n-1) \sim n^2$

Best Case Complexity: $O(n)$

- When the array is already sorted, the outer loop runs for n number of times whereas the inner loop does not run at all. So, there are only n number of comparisons. Thus, complexity is linear.

Average Case Complexity: $O(n^2)$

- It occurs when the elements of an array are in jumbled order (neither ascending nor descending).

Space Complexity

Space complexity is $O(1)$ because an extra variable key is used.

Insertion Sort Practical Implementation:

- The insertionSort function takes an array arr as input. It first calculates the length of the array (n). If the length is 0 or 1, the function returns immediately as an array with 0 or 1 element is considered already sorted.
- For arrays with more than one element, We start with second element of the array as first element in the array is assumed to be sorted.
- Compare second element with the first element and check if the second element is smaller then swap them.
- Move to the third element and compare it with the first two elements and put at its correct position
- Repeat until the entire array is sorted.

```
def insertionSort(a):
    n = len(a) # Get the length of the array

    if n <= 1:
        return # If the array has 0 or 1 element, it is already sorted, so return

    for i in range(1, n): # Iterate over the array starting from the second element
        key = a[i] # Store the current element as the key to be inserted in the right position
        j = i-1
        while j >= 0 and key < a[j]: # Move elements greater than key one position ahead
            a[j+1] = a[j] # Shift elements to the right
            j -= 1
        arr[j+1] = key # Insert the key in the correct position

# Sorting the array [12, 11, 13, 5, 6] using insertionSort
a = [12, 11, 13, 5, 6]
insertionSort(a)
print("Insertion Sorted array:", a)
```

```

def insertionSort(a):
    n = len(a)

    if n <= 1:
        return

    for i in range(1, n):
        key = a[i]
        j = i-1
        while j >= 0 and key < a[j]:
            a[j+1] = a[j]
            j -= 1
        arr[j+1] = key

# Sorting the array [12, 11, 13, 5, 6] using insertionSort
a = [12, 11, 13, 5, 6]
insertionSort(a)
print("Insertion Sorted array:", a)

```

Output:

Insertion Sorted array: [12, 12, 12, 12, 13]

```

def insertionSort(a):
    n = len(a)

    if n <= 1:
        return

    for i in range(1, n):
        key = a[i]
        j = i-1
        print("Pass=",i)
        print("Key =",key)
        while j >= 0 and key < a[j]:
            a[j+1] = a[j]
            j =j - 1
            print("Array: ", a)
        arr[j+1] = key
        print("Output of Pass",a)
        print()

```

```

# Sorting the array [12, 11, 13, 5, 6] using insertionSort
a = [12, 11, 13, 5, 6]

```

```
insertionSort(a)
print("Insertion Sorted array:", a)
```

Output:

Pass= 1

Key = 11

Array: [12, 12, 13, 5, 6]

Output of Pass [12, 12, 13, 5, 6]

Pass= 2

Key = 13

Output of Pass [12, 12, 13, 5, 6]

Pass= 3

Key = 5

Array: [12, 12, 13, 13, 6]

Array: [12, 12, 12, 13, 6]

Array: [12, 12, 12, 13, 6]

Output of Pass [12, 12, 12, 13, 6]

Pass= 4

Key = 6

Array: [12, 12, 12, 13, 13]

Array: [12, 12, 12, 12, 13]

Array: [12, 12, 12, 12, 13]

Array: [12, 12, 12, 12, 13]

Output of Pass [12, 12, 12, 12, 13]

Insertion Sorted array: [12, 12, 12, 12, 13]